

Matricola: _____

Nome: _____

Cognome: _____

ESAME Programmazione Avanzata e Parallela

29 giugno 2026

L'esame consiste in due parti:

- La **prima parte** consiste in 10 domande a risposta multipla sugli argomenti del corso. Ogni domanda può ricevere un punteggio massimo di *due* punti. Affinché una risposta sia considerata valida la scelta *deve essere motivata* in modo breve (non saranno infatti parte della consegna eventuali fogli “di brutta” o aggiuntivi). Una risposta errata o non motivata riceverà *zero* punti. *Il punteggio minimo è 12 punti su 20.*
- La **seconda parte** consiste in due esercizi di programmazione. Ogni esercizio può avere un punteggio massimo di *cinque* punti. Potete commentare brevemente il codice per chiarire le vostre scelte implementative. *Il punteggio minimo è 6 punti su 10.*

IMPORTANTE: *Per raggiungere la sufficienza è necessario raggiungere il punteggio minimo in entrambe le parti.*

Parte 1

Domanda 1

Si supponga di avere il seguente file header:

```
#ifndef __LIST_H
#define __LIST_H

struct list {
    int val;
    struct list * next;
};
struct list * make_empty_list();
struct list * push_front(struct list * lst, int val);
void deallocate_list(struct list * lst);

#endif
```

Per rendere corretto il file header quale delle seguenti modifiche dovrebbe essere compiuta?

☐ Nulla, il file header è già corretto

☐ sostituire `#ifdef` con `#ifndef`

☐ dichiarare sia la struttura che le funzioni come `extern`

☐ aggiungere `#include <stdlib.h>` in quanto vi sono delle funzioni che necessitano di `malloc` e `free`

Domanda 2

Si supponga di avere questo frammento di codice C:

```
while (i < n) {  
    if (v[i]%2 == 1) {  
        a += v[i];  
    } else {  
        b += v[i];  
    }  
    i++;  
}
```

e si assuma di avere variabili `v`, `i`, `a`, `b` e `n` definite e del tipo corretto. La variabile `v` punta a un vettore di `int` di `n` elementi generati casualmente tra il minimo e il massimo valore rappresentabile.

Quale delle seguenti è la versione branchless del codice precedente? La versione branchless migliorerebbe le prestazioni? Si assuma che il compilatore non ottimizzi.

☐

```
int c = (v[i]%2 == 1);  
a += c*v[i];  
b += (1-c)*v[i];
```


Migliorerebbe le prestazioni

☐

```
int c = (v[i]%2 == 1);  
c ? a += v[i] : b += v[i];
```


Migliorerebbe le prestazioni

☐

```
int c = (v[i]%2 == 1);  
a += c*v[i];  
b += (1-c)*v[i];
```


Non migliorerebbe le prestazioni

☐

```
int c = (v[i]%2 == 1);  
c ? a += v[i] : b += v[i];
```


Non migliorerebbe le prestazioni

Domanda 3

Si considerino le seguenti strutture:

```
struct A {  
    int * u;  
    int * v;  
    int * w;  
};
```

```
struct B {  
    int u;  
    int v;  
    int w;  
};
```

E si supponga di avere un vettore di `struct B` di lunghezza n e una singola `struct A` con u , v e w che puntano a vettori di lunghezza n . Quale delle seguenti affermazioni è errata?

- | | |
|---|--|
| ☐ Per sommare tutti i valori di v l'utilizzo della <code>struct A</code> ha migliore località di memoria | ☐ Dato un indice i , accedere all' i -esimo valore per u , v e w ha migliore località di memoria per il vettore di <code>struct B</code> |
| ☐ L'occupazione di memoria di <code>struct A</code> sarà circa tre volte superiore all'occupazione di memoria del vettore di <code>struct B</code> | ☐ Dato un indice i , è indifferente in termini di località di memoria accedere all' i -esimo valore per u tramite <code>struct A</code> o il vettore di <code>struct B</code> |
-
-
-

Domanda 4

Si considerino le estensioni vettoriali che utilizzano registri vettoriali di 128bit (e.g., NEON e SSE). Quale delle seguenti affermazioni è corretta?

- | | |
|---|--|
| ☐ Dato che ogni registro vettoriale può contenere al massimo 4 numeri floating point a precisione singola, le istruzioni possono essere eseguite al massimo su 4 core contemporaneamente | ☐ Tutte le estensioni che utilizzano la stessa dimensione dei registri vettoriali supportano le stesse istruzioni e sono quindi interoperabili tra di loro in modo automatico |
| ☐ Dato che il registro può contenere fino a 128bit le operazioni di somma e prodotto non andranno in overflow ma potranno sfruttare i bit aggiuntivi | ☐ Una istruzione può svolgere la stessa operazione su 4 numeri floating point a precisione singola |
-
-
-

Domanda 5

Quale delle seguenti affermazioni su `mmap` è *errata*?

- | | |
|---|--|
| ☐ È possibile effettuare il mapping di un file con dimensione maggiore della memoria fisica ma inferiore a quello della memoria virtuale | ☐ Se viene mappato un file di dimensione maggiore della memoria fisica non è comunque possibile accedere a tutto il contenuto del file |
| ☐ L'accesso al file può essere effettuato dereferenziando il puntatore ritornato da <code>mmap</code> | ☐ Se salviamo dei puntatori su un file mappato in memoria con <code>mmap</code> non è detto che ad una esecuzione successiva del programma questi puntino ancora a indirizzi validi |
-
-
-

Domanda 6

Dato il seguente codice C facente uso di OpenMP:

```
float f(float (*g) (float), float * M, float * v, int n)
{
    float s = 0;
    #TODO1
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            v[i] = M[i*n + j];
        }
    }
    #TODO2
    for (int i = 0; i < n; i++) {
        v[i] = g(v[i]);
        s += v[i];
    }
    return s;
}
```

Si supponga che M sia una matrice $n \times n$, e v un vettore di n elementi. Si assuma inoltre che g sia thread-safe. Al posto di TODO1 e TODO2 che codice può essere inserito affinché il programma funzioni correttamente?

- | | |
|---|--|
| ☐ #pragma omp parallel for collapse(2)
per TODO1 e
#pragma omp critical per TODO2 | ☐ #pragma omp parallel per TODO1 e anche per TODO2 |
| ☐ #pragma omp parallel for per TODO1 e
#pragma omp parallel for
reduction(+:s) per TODO2 | ☐ #pragma omp parallel for collapse(2)
per TODO1 e
#pragma omp parallel for
application(g:+) |
-
-
-

Domanda 7

Dato il seguente codice Python:

```
class A:
```

```
    def __init__(self, f):
        self.f = f
        self.d = {}
```

```
class B(A):
```

```
    def __setitem__(self, k, v):
        self.d[k] = v
```

```
    def __getitem__(self, k):
        return self.f(self.d[k])
```

```
class C(A):
```

```
    def __setitem__(self, k, v):
        self.d[self.f(k)] = v
```

```
    def __getitem__(self, k):
        return self.d[self.f(k)]
```

```
x = B(lambda x: x+1)
```

```
x[2] = 3
```

```
y = C(lambda y: y+1)
```

```
y[2] = 3
```

```
x[2] == y[2]
```

Quale delle seguenti affermazioni è corretta riguardo all'operazione `x[2] == y[2]`?

- | | |
|---|---|
| ☐ L'operazione ritornerà False in quanto l'istanza di B modifica il valore ritornato da <code>__getitem__</code> | ☐ L'operazione ritornerà True dato che entrambi gli oggetti hanno impostato il valore 3 per l'indice 2 |
| ☐ L'operazione ritornerà True anche se l'istanza di C accederà a un indice diverso dall'istanza di B | ☐ L'operazione ritornerà False in quanto l'istanza di C accede a un indice diverso dall'istanza di B |
-
-
-

Domanda 8

Dato il seguente codice Python:

```
def f(g, h):  
    def k(p):  
        return lambda x: g(h(p, x), p(x))  
    return k  
  
f1 = f(lambda x, y: x + y,  
        lambda k, x: k(k(x)))  
  
f2 = f1(lambda x: x**2)
```

Quale delle seguenti affermazioni descrive correttamente la funzione f2?

- ☐ f2 è di un tipo numerico (e.g., int o float) ☐ f2 è una funzione che computa $x^4 + x^2$
- ☐ f2 è una funzione che computa $(x^2 + x)^2$ ☐ il codice che definisce f2 non è sintatticamente corretto dato che necessita di un ulteriore argomento
-
-
-

Domanda 9

Dato il seguente codice Python:

```
def deco(n):  
    def f(k):  
        def h(*args, **kwargs):  
            lst = [x + n for x in args]  
            return k(*lst, **kwargs)  
        return h  
    return f  
  
@deco(3)  
def k(a, b, c):  
    return a**3 + b**2 + c
```

Quale è l'effetto del decoratore sulla funzione k?

- | | |
|---|---|
| ☐ Trasforma la funzione n in modo che accetti una lista di n argomenti | ☐ Trasforma la funzione k incrementando il valore che ritorna di n |
| ☐ Trasforma la funzione k incrementando il suo primo argomento di n | ☐ Trasforma la funzione k incrementando ciascuno dei suoi argomenti di n |
-
-
-

Domanda 10

Dato il seguente codice Python:

```
def f(v, a, b):  
    for x in v:  
        if x < a:  
            yield a  
        elif x > b:  
            yield b  
        else:  
            yield x
```

```
h = f([7, 8, 3, 2, 5, 1, 6], 2, 4)  
w = [x for x in h]
```

Quale affermazione è corretta rispetto a w ?

- | | |
|---|---|
| ☐ Il generatore non può essere utilizzato per costruire w | ☐ w conterrà solo i valori della lista iniziale che erano compresi tra 2 e 4 |
| ☐ w conterrà tutti i valori della lista passata come argomento in aggiunta a delle copie aggiuntive di 2 e 4 | ☐ w conterrà i valori della lista iniziale con i valori fuori dall'intervallo tra 2 e 4 rimpiazzati da 2 o 4 |
-
-
-

Parte 2

Esercizio 1

Si completi la seguente funzione C che ha i seguenti argomenti:

- `g` è un puntatore a funzione che prende due argomenti: un indice di colonna e il numero di colonne di una matrice. Ritorna un nuovo indice di colonna;
- `Min` è un puntatore a una matrice $n \times n$ di `float`;
- `Mout` è un puntatore a una matrice $n \times n$ di `float` inizialmente tutti zero;
- `n` è un intero che indica il lato della matrice.

La funzione deve:

- Riempire la matrice `Mout` con i valori di `Min` in cui l'indice di riga rimane invariato ma l'indice di colonna di `Mout` è dato da `g`. Ovvero, il valore in posizione (i, j) in `Min` deve essere scritto in posizione $(i, g(j, n))$ su `Mout`. Si assuma `g` produca una permutazione degli indici di colonna.
- Viene ritornata la somma dei valori contenuti nella diagonale di `Mout` dopo che è stata riempita.

Il riempimento della matrice `Mout` e il calcolo della somma dei valori sulla diagonale devono essere parallelizzati con OpenMP in modo che il parallelismo sia adeguatamente sfruttato (i.e., non facendo svolgere le operazioni a un solo thread).

[illegible]

Esercizio 2

Si scriva il metodo `__call__` della classe A che, dato un valore x compone tutte le funzioni in `lst` nell'ordine in cui appaiono nella lista. Quindi, per esempio, se `lst` è $[f_1, f_2, f_3]$, il metodo `__call__` con argomento x ritornerà $f_3(f_2(f_1(x)))$. Nel caso la lista sia vuota il metodo deve comportarsi come la funzione identità.

class A:

```
def __init__(self):  
    self.lst = []
```

```
def append(self, f):  
    self.lst.append(f)
```

```
def __call__(self, x):
```
